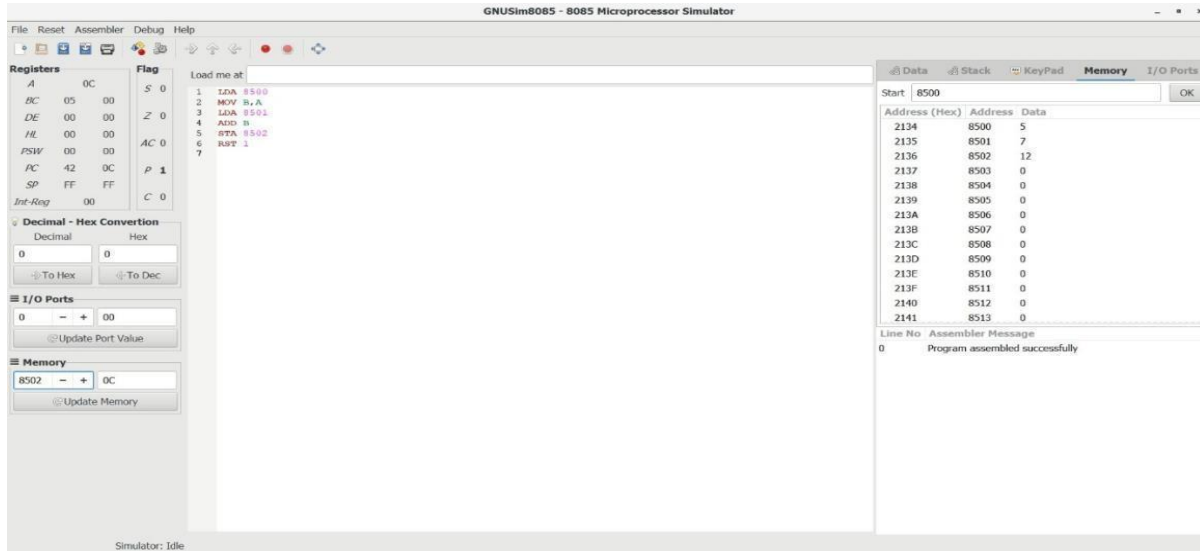


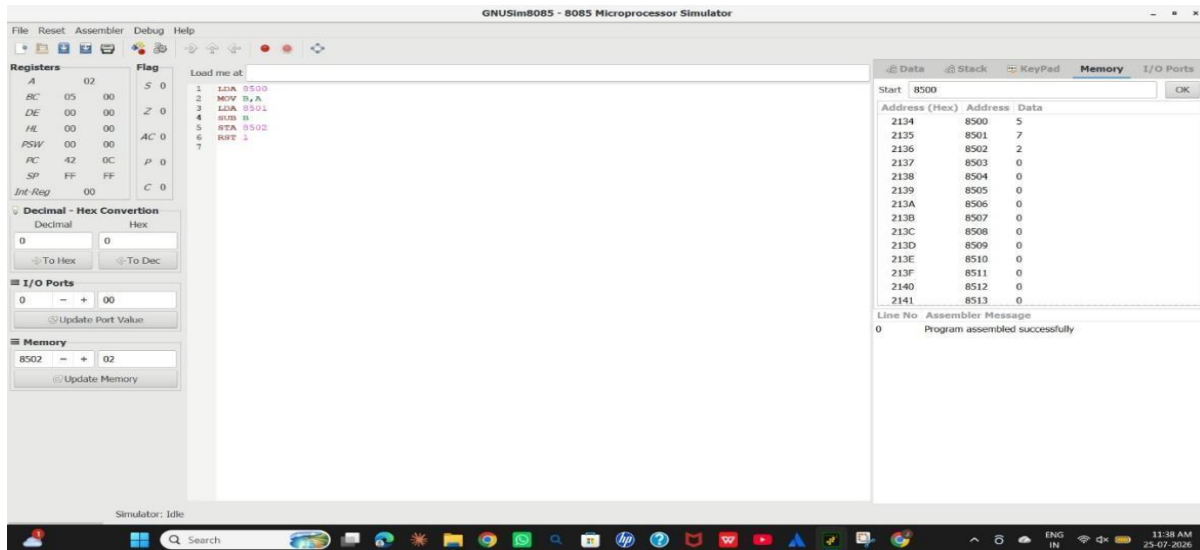
CSA1204 COMPUTER ARCHITECTURE

8085 LAB PROGRAMS

Q1. Addition of 2 Numbers



Q2. Subtraction of 2 Numbers



Q3. Addition With Carry (ADC)

The screenshot shows the GNUSim8085 - 8085 Microprocessor Simulator interface. The main window displays the assembly code for an addition with carry (ADC) operation. The registers and flags are shown on the left, and the memory dump is on the right.

Registers:

Register	Value
A	3D
BC	19 00
DE	00 00
HL	00 00
PSW	00 00
PC	42 0D
SP	FF FF
Int-Reg	00

Flags:

Flag	Value
S	0
Z	0
AC	0
P	0
C	0

Assembly Code:

```
1 LDA 2500
2 MOV B,A
3 LDA 2501
4 STC
5 ADC B
6 STA 2502
7 HLT
8
```

Memory Dump:

Address (Hex)	Address	Data
09C4	2500	25
09C5	2501	35
09C6	2502	61
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Assembler Message:

```
Line No. Assembler Message
0 Program assembled successfully
```

Q4. Subtraction With Borrow (SBB)

The screenshot shows the GNUSim8085 - 8085 Microprocessor Simulator interface. The main window displays the assembly code for a subtraction with borrow (SBB) operation. The registers and flags are shown on the left, and the memory dump is on the right.

Registers:

Register	Value
A	09
BC	19 00
DE	00 00
HL	00 00
PSW	00 00
PC	42 0D
SP	FF FF
Int-Reg	00

Flags:

Flag	Value
S	0
Z	0
AC	0
P	1
C	0

Assembly Code:

```
1 LDA 2500
2 MOV B,A
3 LDA 2501
4 STC
5 SBB B
6 STA 2502
7 HLT
8
```

Memory Dump:

Address (Hex)	Address	Data
09C4	2500	25
09C5	2501	35
09C6	2502	9
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Assembler Message:

```
Line No. Assembler Message
0 Program assembled successfully
```

Q5. Multiplication of 2 Numbers

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

Register	Value
A	32
BC	0A 00
DE	00 00
HL	00 00
PSW	00 00
PC	42 13
SP	FF FF
Int-Reg	00

Flag

Flag	Value
S	0
Z	1
AC	0
P	1
C	0

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

0 - + 00

Update Port Value

Memory

2501 - + 05

Update Memory

Load me at:

```
1 LDA 2500
2 MOV B,A
3 LDA 2501
4 MOV C,A
5 MVI A,00
6 LOOP: ADD B
7 DCR C
8 JNZ LOOP
9 STA 2502
10 HLT
```

Simulator: Idle

30°C Mostly cloudy

Search

ENG IN

13:31 28-07-2026

Data Stack KeyPad Memory I/O Ports

Start: 2500

Address (Hex)	Address	Data
09C4	2500	10
09C5	2501	5
09C6	2502	50
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No. Assembler Message

0 Program assembled successfully

Q6. Division of Two Numbers

GNUSim8085 - 8085 Microprocessor Simulator

File Reset Assembler Debug Help

Registers

Register	Value
A	05
BC	0A 02
DE	05 00
HL	00 00
PSW	00 00
PC	42 1C
SP	FF FF
Int-Reg	00

Flag

Flag	Value
S	0
Z	1
AC	1
P	1
C	1

Decimal - Hex Conversion

Decimal: 0 Hex: 0

I/O Ports

0 - + 00

Update Port Value

Memory

2501 - + 02

Update Memory

Load me at:

```
1 LDA 2500
2 MOV B,A
3 LDA 2501
4 MOV C,A
5 MVI D,00
6 MOV A,B
7 LOOP: RRB C
8 JC STOP
9 INR D
10 JND LOOP
11 STOP: ADD C
12 STA 2503
13 MOV A,D
14 STA 2502
15 HLT
```

Simulator: Idle

30°C Mostly cloudy

Search

ENG IN

01:42 PM 28-07-2026

Data Stack KeyPad Memory I/O Ports

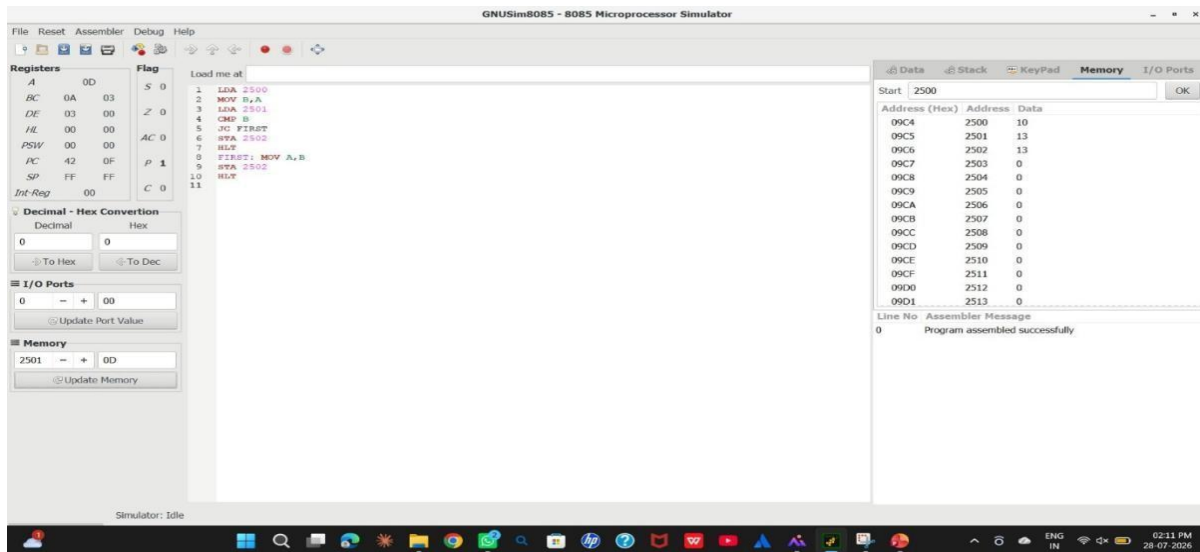
Start: 2500

Address (Hex)	Address	Data
09C4	2500	10
09C5	2501	2
09C6	2502	5
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

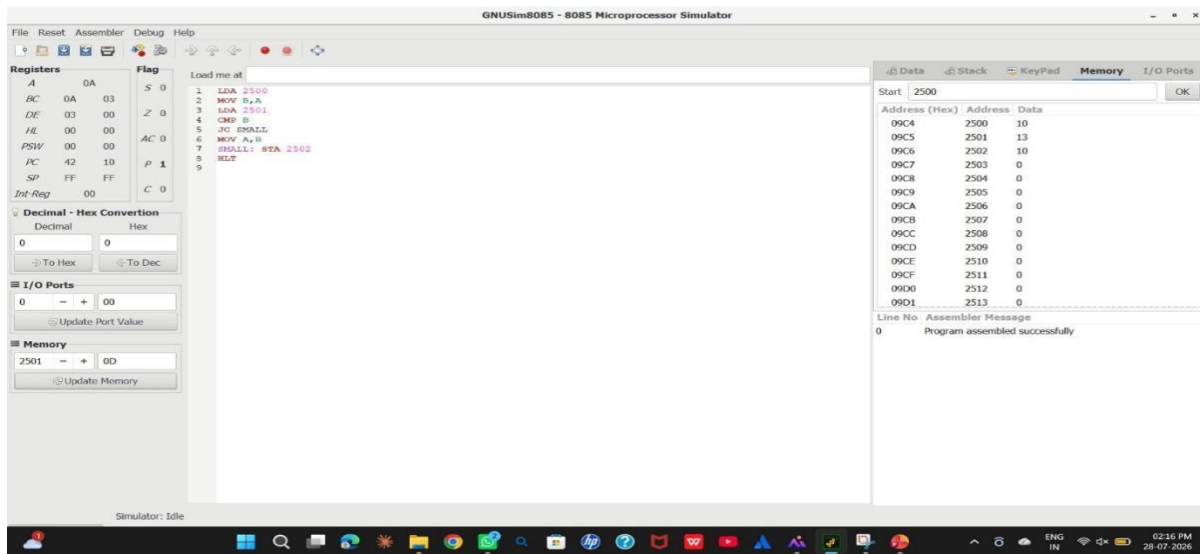
Line No. Assembler Message

0 Program assembled successfully

Q7. Greatest of 2 Numbers



Q8. Smallest of 2 Numbers



Q9. Factorial of a Number

Flag

S 0

Z 1

DD AC 0

00 P 1

FF C 0

Conversion

Hex

0

< To Dec

00

Port Value

05

Memory

Load me at

```
1 LDA 2500
2 MOV C,A
3 MVI B,01H
4
5 FACT: MOV A,C
6 CPI 00H
7 JZ DONE
8
9 MOV D,C
10 MOV A,B
11 MOV E,A
12 MVI A,00H
13
14 MUL: ADD E
15 DCR D
16 JNZ MUL
17
18 MOV B,A
19 DCR C
20 JMP FACT
21
22 DONE: MOV A,B
23 STA 2501
24 HLT
```

Data

Stack

KeyPad

Me

Start 2500

Address (Hex)	Address	Data
09C4	2500	5
09C5	2501	120
09C6	2502	2
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q10. Packed BCD to Binary Conversion

S 0

Z 0

AC 1

P 1

C 0

Conversion

Hex

< To Dec

Value

Memory

```
1 LDA 2500
2 MOV B,A
3 ANI 0FH
4 MOV E,A
5
6 MOV A,B
7 ANI 0F0H
8 RRC
9 RRC
10 RRC
11 RRC
12 MOV D,A
13
14 MVI C,0AH
15 MVI A,00H
16
17 LOOP: ADD D
18 DCR C
19 JNZ LOOP
20
21 ADD E
22 STA 2501
23 HLT
```

Start 2500

Address (Hex)	Address	Data
09C4	2500	45
09C5	2501	33
09C6	2502	2
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q11. GCD (HCF) of Two Numbers

The screenshot shows an 8085 assembly program for finding the GCD of two numbers. The program is loaded at address 02. The assembly code is as follows:

```
02 02 S 0 1 LDA 2500
18 18 Z 1 2 MOV B,A
09 DD AC 0 3 LDA 2501
00 00 P 1 4 MOV C,A
FF FF C 0 5
10 10 6 LOOP: MOV A,B
11 11 7 CMP C
12 12 8 JZ DONE
13 13 9 JC SECOND
14 14 10 SUB C
15 15 11 MOV B,A
16 16 12 JMP LOOP
17 17 13
18 18 14 SECOND: MOV A,C
19 19 15 SUB B
20 20 16 MOV C,A
21 21 17 JMP LOOP
22 22 18
23 23 19 DONE: MOV A,B
24 24 20 STA 2502
25 25 21 HLT
```

The program uses the Euclidean algorithm to find the GCD. It starts by loading two numbers from memory addresses 2500 and 2501 into registers B and C. It then enters a loop where it repeatedly subtracts the smaller number from the larger one until they are equal. The result is then stored at address 2502.

The memory dump shows the following data:

Address (Hex)	Address	Data
09C4	2500	6
09C5	2501	8
09C6	2502	2
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The assembler message shows: 0 Program assembled successfully.

Q12. LCM of Two Numbers

The screenshot shows an 8085 assembly program for finding the LCM of two numbers. The program is loaded at address 08. The assembly code is as follows:

```
08 08 S 0 1 LDA 2500
18 18 Z 1 2 MOV B,A
DD DD AC 0 3 LDA 2501
00 00 P 1 4 MOV C,A
FF FF C 0 5
10 10 6 MOV D,B
11 11 7 MOV E,C
12 12 8
13 13 9 LOOP: MOV A,D
14 14 10 CMP E
15 15 11 JZ DONE
16 16 12 JC ADDD
17 17 13
18 18 14 MOV A,E
19 19 15 ADD C
20 20 16 MOV E,A
21 21 17 JMP LOOP
22 22 18
23 23 19 ADDD: MOV A,D
24 24 20 ADD B
25 25 21 MOV D,A
26 26 22 JMP LOOP
27 27 23
28 28 24 DONE: MOV A,D
29 29 25 STA 2502
30 30 26 HLT
```

The program uses the formula LCM = (A * B) / GCD. It first finds the GCD using the Euclidean algorithm (as in Q11). Then, it calculates the LCM by multiplying the two numbers and dividing the result by the GCD. The result is then stored at address 2502.

The memory dump shows the following data:

Address (Hex)	Address	Data
09C4	2500	6
09C5	2501	8
09C6	2502	24
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The assembler message shows: 0 Program assembled successfully.

Q13. Smallest Number in an Array

Load me at

```
1 LXI H,2501
2 MOV B,M
3 MOV A,M
4 MOV D,A
5 DCR B
6
7 LOOP: INX H
8 MOV A,M
9 CMP D
10 JNC SKIP
11 MOV D,A
12
13 SKIP: DCR B
14 JNZ LOOP
15
16 MOV A,D
17 STA 2600
18 HLT
```

Data Stack KeyPad Memory

Start 2500

Address (Hex)	Address	Data
09C4	2500	5
09C5	2501	25
09C6	2502	10
09C7	2503	45
09C8	2504	15
09C9	2505	30
09CA	2506	5
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q14. Largest Number in an Array

Flag

S 0

Z 1

AC 0

P 1

C 1

Inversion

Hex

To Dec

0

t Value

E

emory

Load me at

```
1 LXI H,2501
2 MOV B,M
3 MOV A,M
4 MOV D,A
5 DCR B
6
7 LOOP: INX H
8 MOV A,M
9 CMP D
10 JC SKIP
11 MOV D,A
12
13 SKIP: DCR B
14 JNZ LOOP
15
16 MOV A,D
17 STA 2506
18 HLT
```

Data Stack KeyPad Memory

Start 2500

Address (Hex)	Address	Data
09C4	2500	5
09C5	2501	25
09C6	2502	10
09C7	2503	45
09C8	2504	15
09C9	2505	30
09CA	2506	45
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q15. Bubble Sort – Descending Order

```
1 LDA 2500
2 DCR A
3 MOV B,A
4
5 OUTER: LXI H,2501
6 MOV C,B
7
8 INNER: MOV A,M
9 MOV D,A
10 INX H
11 MOV A,M
12 MOV E,A
13 MOV A,D
14 CMP E
15 JNC NOSWAP
16
17 MOV M,D
18 DCX H
19 MOV M,E
20 INX H
21
22 NOSWAP: DCR C
23 JNZ INNER
24 DCR B
25 JNZ OUTER
26
27 HLT
```

Start 2500

Address (Hex)	Address	Data
09C4	2500	5
09C5	2501	8
09C6	2502	8
09C7	2503	2
09C8	2504	1
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q16. Bubble Sort – Ascending Order

Flag

S 0

Z 1

AC 0

P 1

C 0

version

Hex

To Dec

value

ory

```
1 LDA 2500
2 DCR A
3 MOV B,A
4
5 OUTER: LXI H,2501
6 MOV C,B
7
8 INNER: MOV A,M
9 MOV D,A
10 INX H
11 MOV A,M
12 MOV E,A
13 MOV A,D
14 CMP E
15 JC NOSWAP
16 JZ NOSWAP
17
18 MOV M,D
19 DCX H
20 MOV M,E
21 INX H
22
23 NOSWAP: DCR C
24 JNZ INNER
25 DCR B
26 JNZ OUTER
27
28 HLT
```

Start 2500

Address (Hex)	Address	Data
09C4	2500	4
09C5	2501	1
09C6	2502	2
09C7	2503	5
09C8	2504	8
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Line No Assembler Message

0 Program assembled successfully

Q17. Check Whether a Number Is Positive

The screenshot shows an 8086 assembly simulator. The assembly code is as follows:

```
1 LDA 2500
2 ANI 80H
3
4 JZ POSITIVE
5
6 MVI A, 01H
7 STA 2501
8 HLT
9
10 POSITIVE: MVI A, 00H
11 STA 2501
12
13 HLT
```

The flag register (F) shows: S 0, Z 1, AC 1, P 1, C 0. The memory window (Memory) shows the following data:

Address (Hex)	Address	Data
09C4	2500	25
09C5	2501	0
09C6	2502	0
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The assembler message window shows: 0 Program assembled successfully.

Q18. Check Whether a Number Is Odd or Even

The screenshot shows an 8086 assembly simulator. The assembly code is as follows:

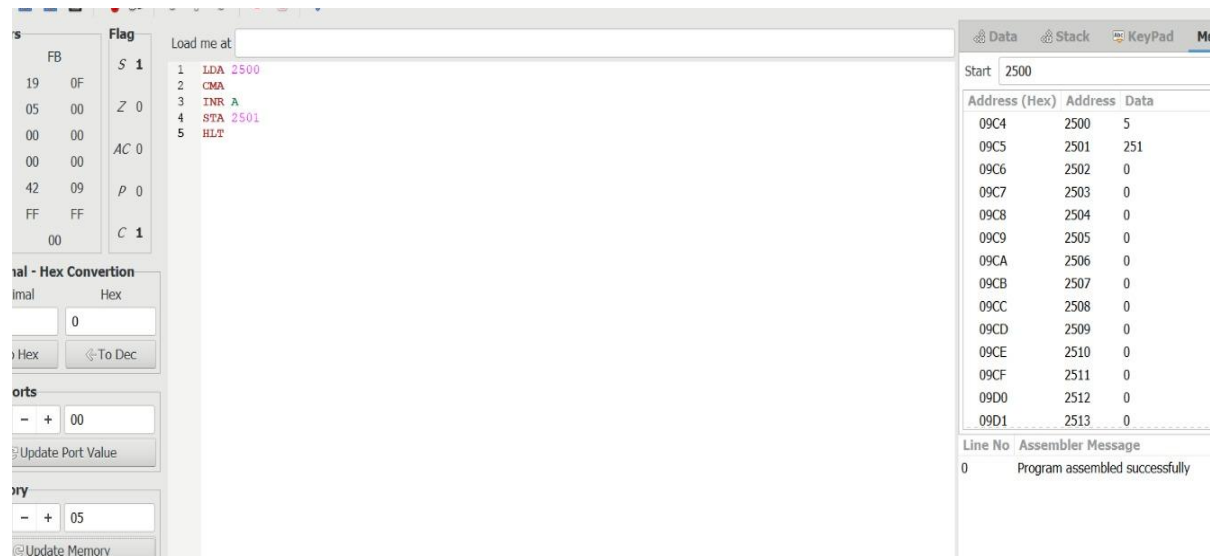
```
1 LDA 2500
2 ANI 01H
3
4 JZ EVEN
5
6 MVI A, 01H
7 STA 2501
8 HLT
9
10 EVEN: MVI A, 00H
11 STA 2501
12
13 HLT
```

The flag register (F) shows: S 0, Z 0, AC 1, P 0, C 0. The memory window (Memory) shows the following data:

Address (Hex)	Address	Data
09C4	2500	9
09C5	2501	1
09C6	2502	0
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The assembler message window shows: 0 Program assembled successfully.

Q19. 2's Complement of a Number



The screenshot shows the 8085 microprocessor simulator interface. The assembly window contains the following code:

```
1 LDA 2500
2 CMA
3 INR A
4 STA 2501
5 HLT
```

The flag register (F) is shown with the following values:

Flag	Value
S	1
Z	0
AC	0
P	0
C	1

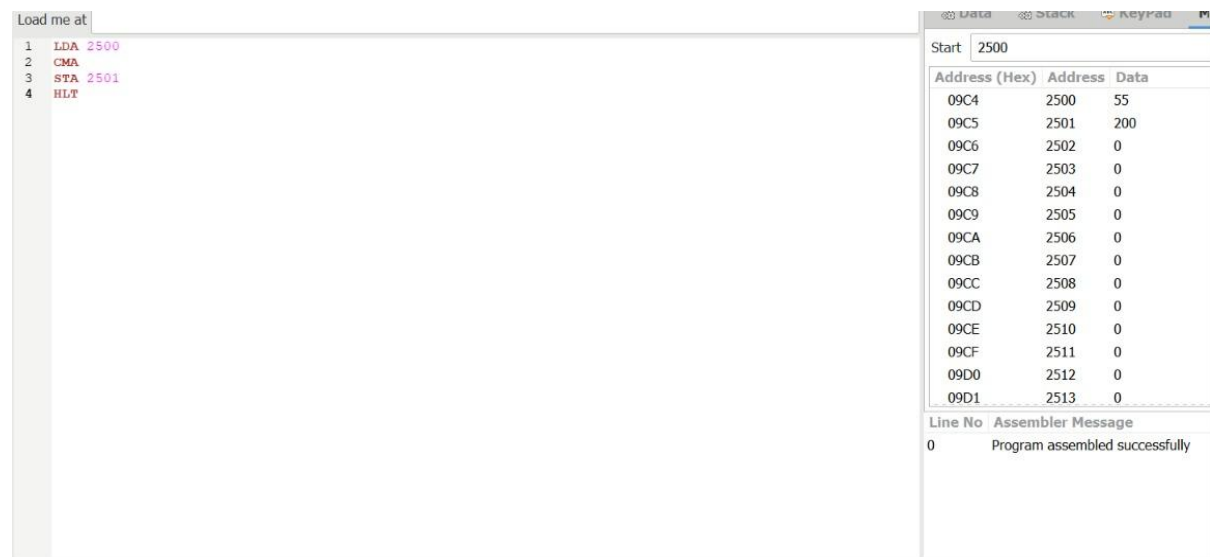
The Data window shows the memory contents:

Address (Hex)	Address	Data
09C4	2500	5
09C5	2501	251
09C6	2502	0
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The Assembler Message window shows:

```
0 Program assembled successfully
```

Q20. 1's Complement of a Number



The screenshot shows the 8085 microprocessor simulator interface. The assembly window contains the following code:

```
1 LDA 2500
2 CMA
3 STA 2501
4 HLT
```

The Data window shows the memory contents:

Address (Hex)	Address	Data
09C4	2500	55
09C5	2501	200
09C6	2502	0
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

The Assembler Message window shows:

```
0 Program assembled successfully
```

Q21. Swap Two Numbers

The screenshot displays a microprocessor simulator interface. On the left, a list of registers (S, Z, AC, P, C) is shown with their current values. The main area contains assembly code for swapping two numbers stored at memory locations 2500 and 2501. The code uses the following instructions:

```
1 LDA 2500
2 MOV B,A
3
4 LDA 2501
5 MOV C,A
6
7 MOV A,B
8 STA 2501
9
10 MOV A,C
11 STA 2500
12
13 HLT
```

On the right, a memory dump table shows the contents of memory locations from 09C4 to 09D1:

Address (Hex)	Address	Data
09C4	2500	15
09C5	2501	25
09C6	2502	0
09C7	2503	0
09C8	2504	0
09C9	2505	0
09CA	2506	0
09CB	2507	0
09CC	2508	0
09CD	2509	0
09CE	2510	0
09CF	2511	0
09D0	2512	0
09D1	2513	0

Below the memory dump, the assembler message window shows:

```
Line No Assembler Message
0 Program assembled successfully
```

Q22. Decimal to Hexadecimal Conversion (8085)

The screenshot shows the GNUSim8085 - 8085 Microprocessor Simulator interface. The title bar indicates the program is titled "GNUSim8085 - 8085 Microprocessor Simulator [Q22. Decimal to Hexadecimal]". The menu bar includes File, Reset, Assembler, Debug, and Help. The Registers window shows the current state of the processor registers:

Register	Value
A	2F
BC	04 00
DE	00 00
HL	85 01
PSW	2F 00
PC	85 0F
SP	FF FF
Int-Reg	00

The Flag window shows the status of the flags:

Flag	Value
S	0
Z	0
AC	0
P	0
C	0

The main assembly code window displays the following program:

```
1 LXI H, 8500H
2 MOV A, M
3 MOV B, A
4 MVI A, 00H
5 MVI C, 0AH
6 MULOOP: ADD B
7 DCR C
8 JNZ MULOOP
9 INX H
10 ADD M
11 STA 8502H
12 HLT
```

The I/O Ports window shows the current port value as 0. The Memory window displays the contents of memory locations from 2134 to 2143:

Address (Hex)	Address	Data
2134	8500	4
2135	8501	7
2136	8502	2F
2137	8503	0
2138	8504	0
2139	8505	0
213A	8506	0
213B	8507	0
213C	8508	0
213D	8509	0
213E	850A	0
213F	850B	0
2140	850C	0
2141	850D	0
2142	850E	0
2143	850F	0

The Assembler Message window shows:

```
Line No Assembler Message
0 Program assembled successfully
```

The status bar at the bottom indicates "Simulator: Idle".

Q23. Addition of Two 16-bit Numbers (8085)

GNUSim8085 - 8085 Microprocessor Simulator [Q23. 16-bit Addition]

File Reset Assembler Debug Help

Registers Flag

A	00	S	0
BC	00 00	Z	1
DE	12 34	AC	0
HL	55 55	P	1
PSW	00 44	C	0
PC	85 12		
SP	FF FF		
Int-Reg	00		

Decimal - Hex Conversion

Decimal: Hex:

To Hex: To Dec:

I/O Ports

0

Update Port Value

Load me at:

```
1 LHLD 8500H
2 XCHG
3 LHLD 8502H
4 MOV A, L
5 ADD E
6 MOV L, A
7 MOV A, H
8 ADC D
9 MOV H, A
10 SHLD 8504H
11 MVI A, 00H
12 ACI 00H
13 STA 8506H
14 HLT
```

Line No Assembler Message

0 Program assembled successfully

Simulator: Idle

Data	Stack	KeyPad	Memory	I/O Ports
Start	8500	OK		
Address (Hex)	Address	Data		
2134	8500	34		
2135	8501	12		
2136	8502	21		
2137	8503	43		
2138	8504	55		
2139	8505	55		
213A	8506	0		
213B	8507	0		
213C	8508	0		
213D	8509	0		
213E	850A	0		
213F	850B	0		
2140	850C	0		
2141	850D	0		
2142	850E	0		
2143	850F	0		

Q24. Subtraction of Two 16-bit Numbers (8085)

GNUSim8085 - 8085 Microprocessor Simulator [Q24. 16-bit Subtraction]

File Reset Assembler Debug Help

Registers Flag

A	44	S	0
BC	00 00	Z	0
DE	56 78	AC	0
HL	44 44	P	1
PSW	44 44	C	0
PC	85 00		
SP	FF FF		
Int-Reg	00		

Decimal - Hex Conversion

Decimal: Hex:

To Hex: To Dec:

I/O Ports

0

Update Port Value

Load me at:

```
1 LHLD 8500H
2 XCHG
3 LHLD 8502H
4 MOV A, E
5 SUB L
6 MOV L, A
7 MOV A, D
8 SBB H
9 MOV H, A
10 SHLD 8504H
11 HLT
```

Line No Assembler Message

0 Program assembled successfully

Simulator: Idle

Data	Stack	KeyPad	Memory	I/O Ports
Start	8500	OK		
Address (Hex)	Address	Data		
2134	8500	78		
2135	8501	56		
2136	8502	34		
2137	8503	12		
2138	8504	44		
2139	8505	44		
213A	8506	0		
213B	8507	0		
213C	8508	0		
213D	8509	0		
213E	850A	0		
213F	850B	0		
2140	850C	0		
2141	850D	0		
2142	850E	0		
2143	850F	0		

Q25. Multiplication of Two 16-bit Numbers (8085)

GNUSim8085 - 8085 Microprocessor Simulator [Q25. 16-bit Multiplication]

File Reset Assembler Debug Help

Registers Flag

A 00 S 0
BC 00 00 Z 1
DE 12 34 AC 0
HL 5B 04 P 1
PSW 00 44 C 0
PC 85 21
SP FF FF
Int-Reg 00

Decimal - Hex Conversion

Decimal Hex
To Hex To Dec

I/O Ports

0
Update Port Value

Load me at 8500

```
1 LXI H, 0000H
2 SHLD 8504H
3 SHLD 8506H
4 LHLD 8502H
5 MOV B, H
6 MOV C, L
7 LOOP: MOV A, B
8 ORA C
9 JZ DONE
10 LHLD 8504H
11 XCHG
12 LHLD 8500H
13 MOV A, E
14 ADD L
15 MOV E, A
16 MOV A, D
17 ADC H
18 MOV D, A
19 XCHG
20 SHLD 8504H
21 JNC SKIP
22 LXI H, 8506H
23 INR M
24 SKIP: DCX B
25 JMP LOOP
26 DONE: HLT
```

Line No Assembler Message
0 Program assembled successfully

Simulator: Idle

Data Stack KeyPad Memory I/O Ports

Start 8500 OK

Address (Hex)	Address	Data
2134	8500	34
2135	8501	12
2136	8502	05
2137	8503	0
2138	8504	04
2139	8505	5B
213A	8506	0
213B	8507	0
213C	8508	0
213D	8509	0
213E	850A	0
213F	850B	0
2140	850C	0
2141	850D	0
2142	850E	0
2143	850F	0

Q26. Division of Two 16-bit Numbers (8085)

GNUSim8085 - 8085 Microprocessor Simulator [Q26. 16-bit Division]

File Reset Assembler Debug Help

Registers Flag

A FF S 1
BC 01 F9 Z 0
DE 01 00 AC 1
HL 01 00 P 0
PSW FF 01 C 1
PC 85 1E
SP FF FF
Int-Reg 00

Decimal - Hex Conversion

Decimal Hex
To Hex To Dec

I/O Ports

0
Update Port Value

Load me at 8500

```
1 LXI H, 8504H
2 MVI M, 00H
3 INX H
4 MVI M, 00H
5 LHLD 8500H
6 XCHG
7 LHLD 8502H
8 LOOP: MOV A, E
9 SUB L
10 MOV C, A
11 MOV A, D
12 SBB H
13 MOV B, A
14 JC DONE
15 MOV E, C
16 MOV D, B
17 LXI H, 8504H
18 INR M
19 JNZ SKIP
20 INX H
21 INR M
22 SKIP: LHLD 8502H
23 JMP LOOP
24 DONE: XCHG
25 SHLD 8506H
26 HLT
```

Line No Assembler Message
0 Program assembled successfully

Simulator: Idle

Data Stack KeyPad Memory I/O Ports

Start 8500 OK

Address (Hex)	Address	Data
2134	8500	32
2135	8501	0
2136	8502	07
2137	8503	0
2138	8504	07
2139	8505	0
213A	8506	01
213B	8507	0
213C	8508	0
213D	8509	0
213E	850A	0
213F	850B	0
2140	850C	0
2141	850D	0
2142	850E	0
2143	850F	0

Q27. Addition of Two 16-bit Numbers (8086)

emu8086 - 8086 Microprocessor Emulator [Q27. 16-bit Addition (8086)]

File Edit Emulate Compile Debug Help

Registers Flags Source Data Segment / Variables

AX	5555	OF	0	01	DATA SEGMENT	Name	Value
BX	4321	DF	0	02	NUM1 DW 1234H	NUM1	1234H
CX	0000	IF	1	03	NUM2 DW 4321H	NUM2	4321H
DX	0000	SF	0	04	RESULT DW ?	RESULT	5555H
SI	0000	ZF	0	05	DATA ENDS		
DI	0000	AF	0	06	CODE SEGMENT		
BP	0000	PF	1	07	ASSUME CS:CODE, DS:DATA		
SP	FFEE	CF	0	08	START:		
CS	0B00			09	MOV AX, DATA		
DS	0AF0			10	MOV DS, AX		
ES	0AF0			11	MOV AX, NUM1		
SS	0B10			12	ADD AX, NUM2		
IP	0016			13	MOV RESULT, AX		
				14	MOV AH, 4CH		
				15	INT 21H		
				16	CODE ENDS		
				17	END START		

Line No Assembler Message

0 Program executed - AH=4CH (terminate)

Emulation complete - program halted (INT 21H, AH=4CH)

Q28. Subtraction of Two 16-bit Numbers (8086)

emu8086 - 8086 Microprocessor Emulator [Q28. 16-bit Subtraction (8086)]

File Edit Emulate Compile Debug Help

Registers Flags Source Data Segment / Variables

AX	4444	OF	0	01	DATA SEGMENT	Name	Value
BX	1234	DF	0	02	NUM1 DW 5678H	NUM1	5678H
CX	0000	IF	1	03	NUM2 DW 1234H	NUM2	1234H
DX	0000	SF	0	04	RESULT DW ?	RESULT	4444H
SI	0000	ZF	0	05	DATA ENDS		
DI	0000	AF	0	06	CODE SEGMENT		
BP	0000	PF	0	07	ASSUME CS:CODE, DS:DATA		
SP	FFEE	CF	0	08	START:		
CS	0B00			09	MOV AX, DATA		
DS	0AF0			10	MOV DS, AX		
ES	0AF0			11	MOV AX, NUM1		
SS	0B10			12	SUB AX, NUM2		
IP	0016			13	MOV RESULT, AX		
				14	MOV AH, 4CH		
				15	INT 21H		
				16	CODE ENDS		
				17	END START		

Line No Assembler Message

0 Program executed - AH=4CH (terminate)

Emulation complete - program halted (INT 21H, AH=4CH)

Q29. Multiplication of Two 16-bit Numbers (8086)

emu8086 - 8086 Microprocessor Emulator [Q29. 16-bit Multiplication (8086)]

File Edit Emulate Compile Debug Help

Registers Flags Source Data Segment / Variables

AX 2468	OF 0	01 DATA SEGMENT	Name Value
BX 0002	DF 0	02 NUM1 DW 1234H	NUM1 1234H
CX 0000	IF 1	03 NUM2 DW 0002H	NUM2 0002H
DX 0000	SF 0	04 RESL DW ?	RESL 2468H
SI 0000	ZF 0	05 RESH DW ?	RESH 0000H
DI 0000	AF 0	06 DATA ENDS	
BP 0000	PF 0	07 CODE SEGMENT	
SP FFEE	CF 0	08 ASSUME CS:CODE, DS:DATA	
CS 0B00		09 START:	
DS 0AF0		10 MOV AX, DATA	
ES 0AF0		11 MOV DS, AX	
SS 0B10		12 MOV AX, NUM1	
IP 0018		13 MOV BX, NUM2	
		14 MUL BX	
		15 MOV RESL, AX	
		16 MOV RESH, DX	
		17 MOV AH, 4CH	
		18 INT 21H	
		19 CODE ENDS	
		20 END START	

Line No Assembler Message

0 Program executed - AH=4CH (terminate)

Emulation complete - program halted (INT 21H, AH=4CH)

Q30. Division of Two 16-bit Numbers (8086)

emu8086 - 8086 Microprocessor Emulator [Q30. 16-bit Division (8086)]

File Edit Emulate Compile Debug Help

Registers Flags Source Data Segment / Variables

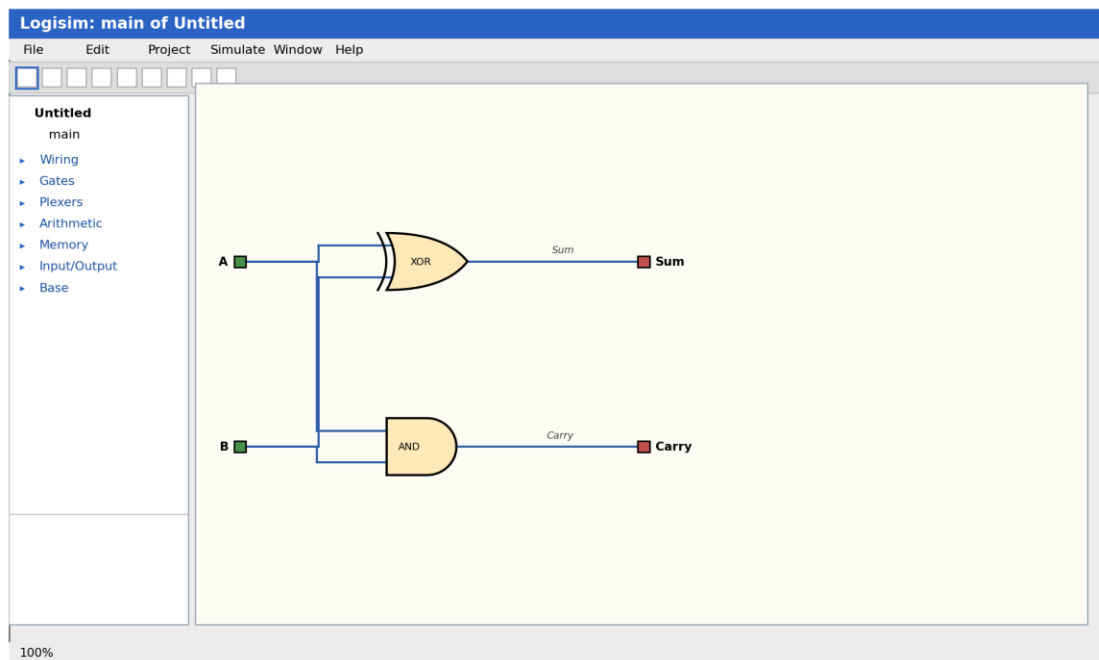
AX 000E	OF 0	01 DATA SEGMENT	Name Value
BX 0007	DF 0	02 DIVIDEND DW 0064H	DIVIDEND 0064H
CX 0000	IF 1	03 DIVISOR DW 0007H	DIVISOR 0007H
DX 0002	SF 0	04 QUOT DW ?	QUOT 000EH
SI 0000	ZF 0	05 REM DW ?	REM 0002H
DI 0000	AF 0	06 DATA ENDS	
BP 0000	PF 0	07 CODE SEGMENT	
SP FFEE	CF 0	08 ASSUME CS:CODE, DS:DATA	
CS 0B00		09 START:	
DS 0AF0		10 MOV AX, DATA	
ES 0AF0		11 MOV DS, AX	
SS 0B10		12 MOV DX, 0000H	
IP 001A		13 MOV AX, DIVIDEND	
		14 MOV BX, DIVISOR	
		15 DIV BX	
		16 MOV QUOT, AX	
		17 MOV REM, DX	
		18 MOV AH, 4CH	
		19 INT 21H	
		20 CODE ENDS	
		21 END START	

Line No Assembler Message

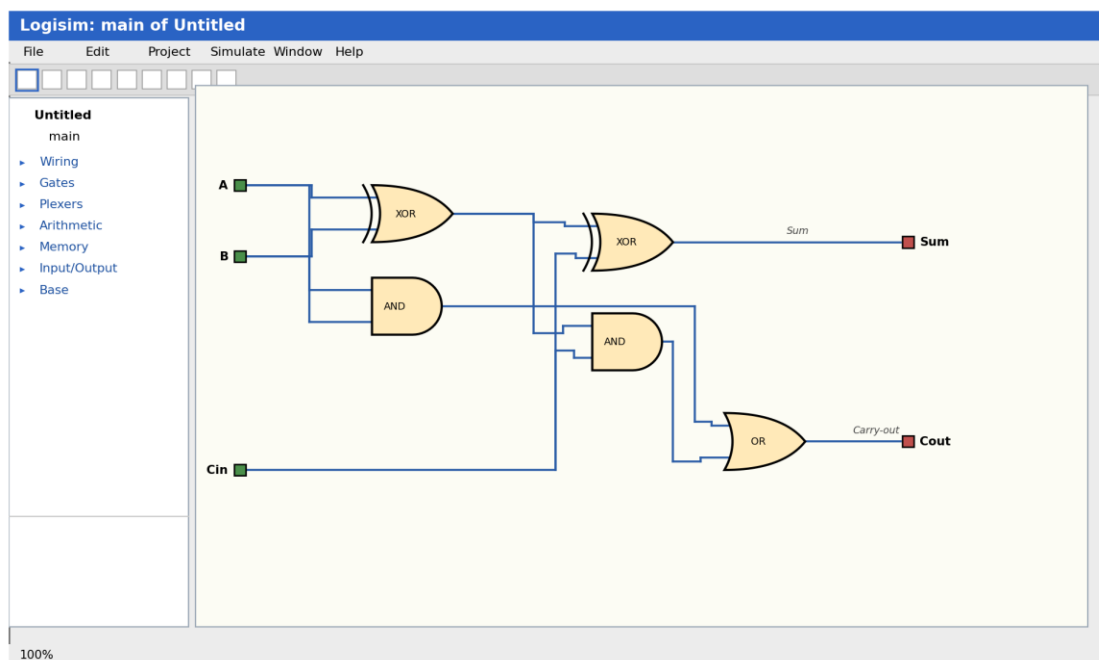
0 Program executed - AH=4CH (terminate)

Emulation complete - program halted (INT 21H, AH=4CH)

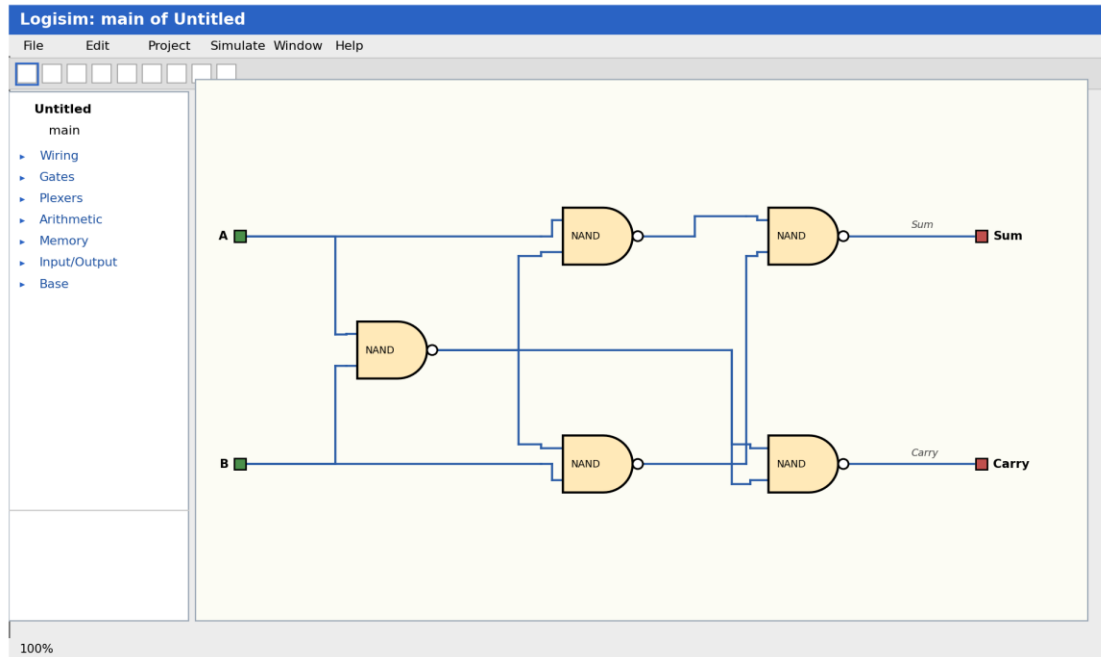
Experiment 31 — Half Adder using Basic Gates



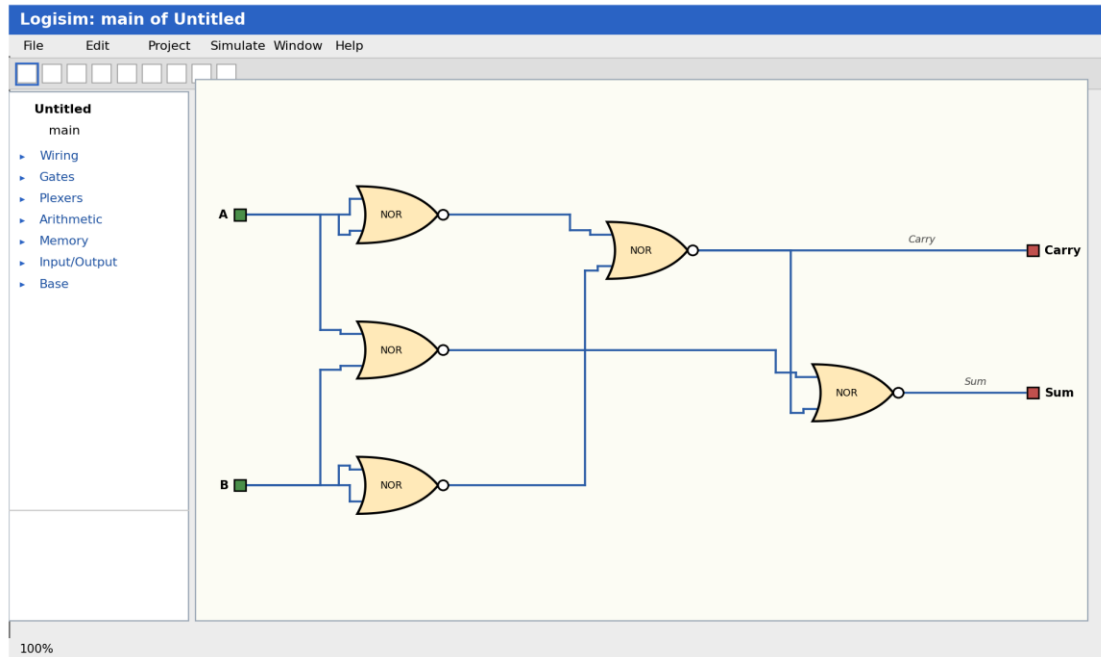
Experiment 32 — Full Adder using Basic Gates



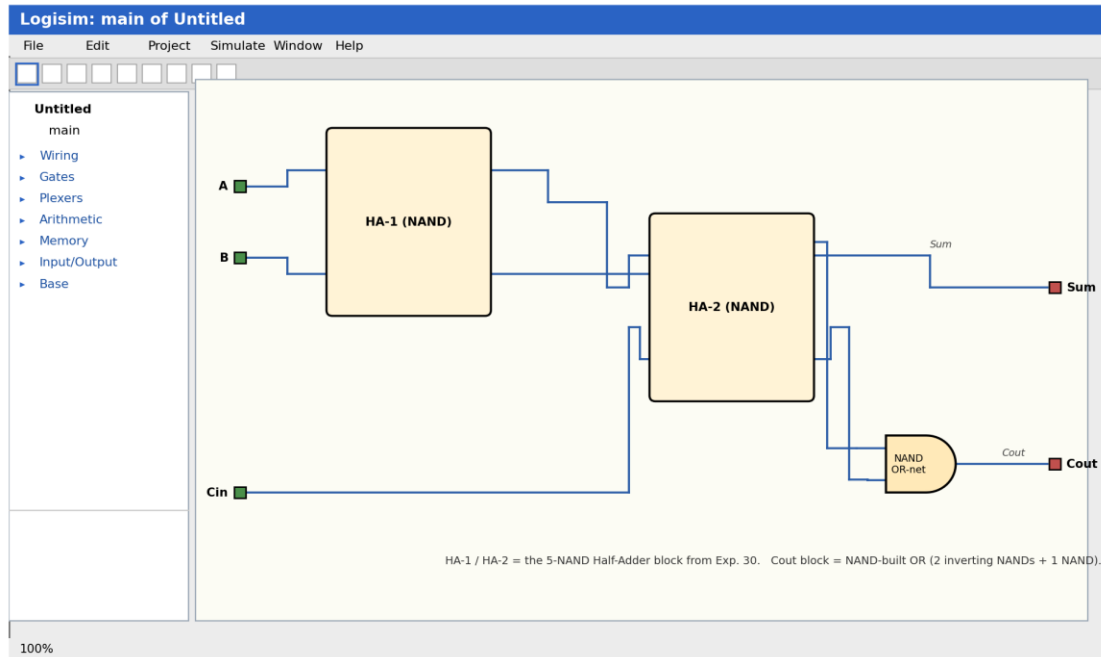
Experiment 33 — Half Adder using NAND Gates Only



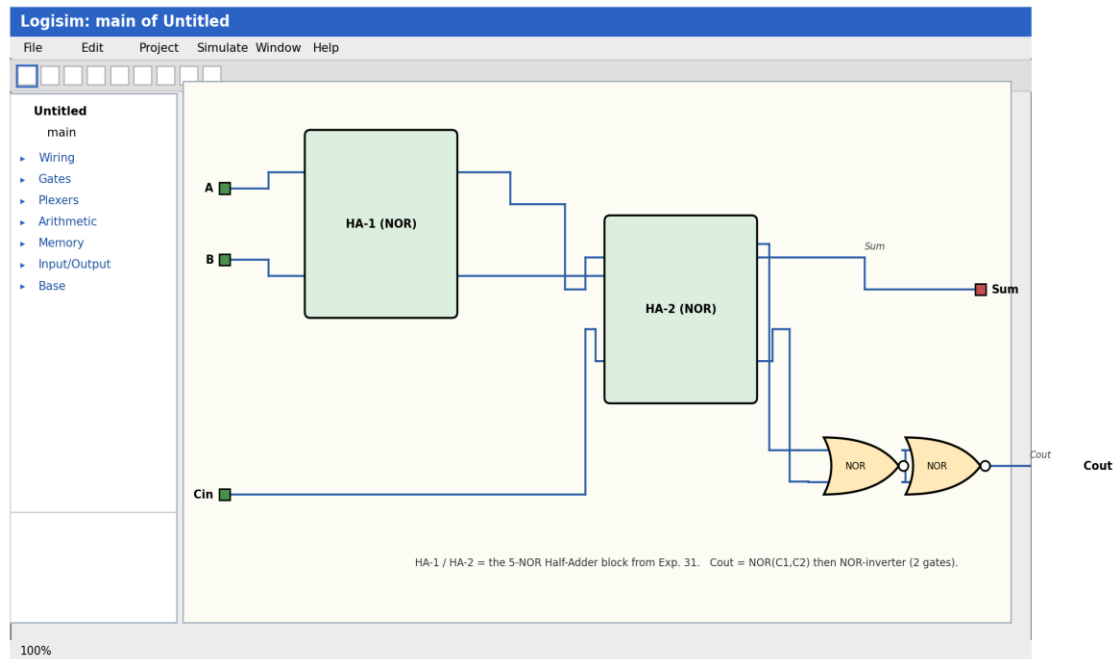
Experiment 34 — 2-bit Half Adder using NOR Gates Only



Experiment 35 — Full Adder using NAND Gates Only



Experiment 36 — Full Adder using NOR Gates Only



Experiment 37— 4-bit Carry Look-Ahead Adder

